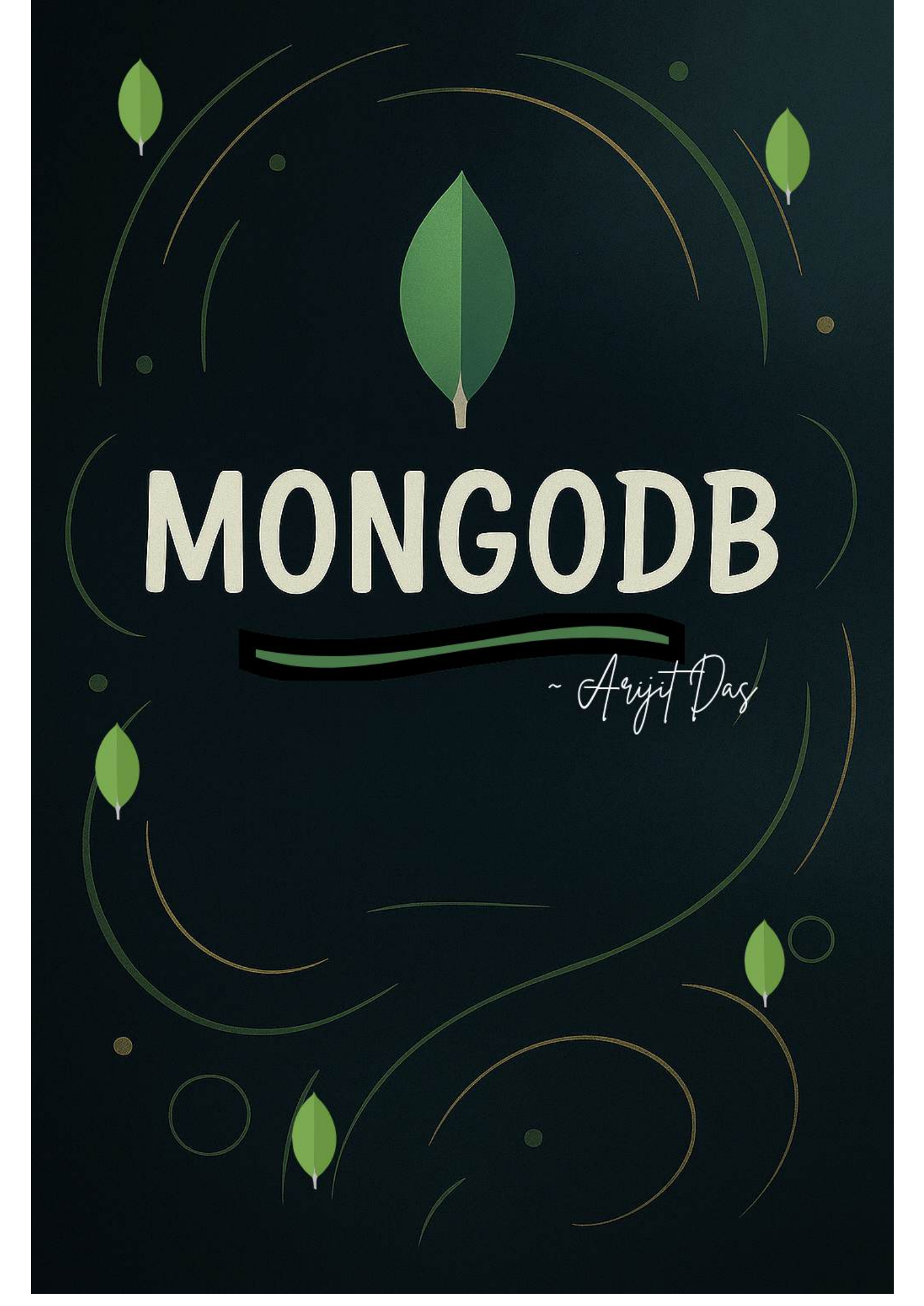


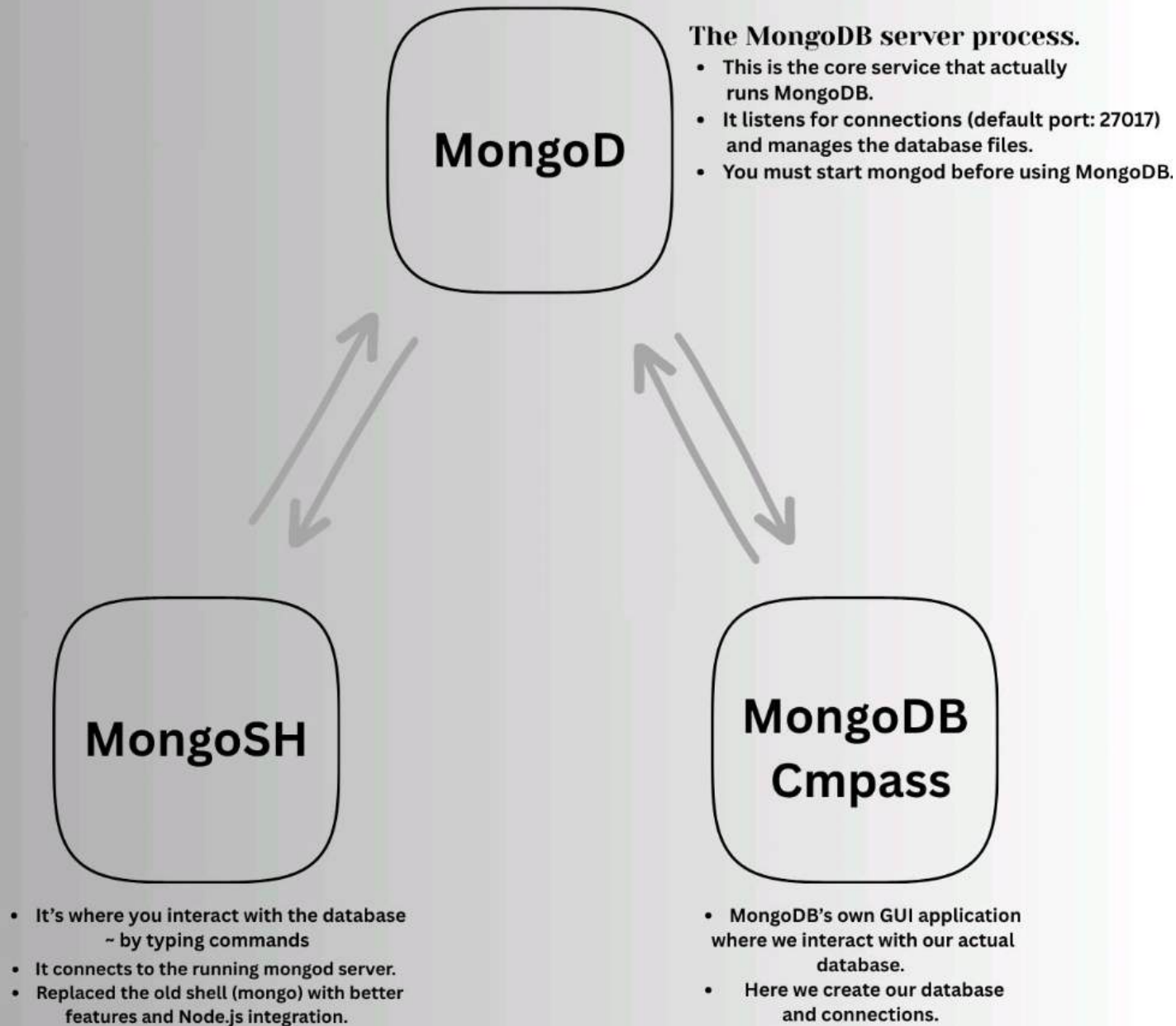


# MONGODB

~ Arijit Das

# MongoDB

## WORKFLOW





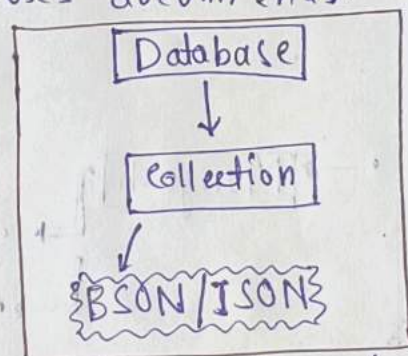
## Introduction

### ■ What is MongoDB?

- ⇒ MongoDB is a NoSQL, documentation-oriented database used to store data in a flexible, JSON-like format (BSON).
- Unlike traditional databases (SQL or PostgreSQL), MongoDB does not use "Tables" or "columns", it uses documents.

### ■ Key Concepts:

- Database: A container for collection data.
- Collection: A group of documents.
- Document: A single record stored in JSON / BSON format.



### ■ Example:

```
{
  "name": "Arizid",
  "age": 20,
  "skills": ['c++', 'Java', 'Python']
}
```

### ■ Why MongoDB?

- Schema-less: Each document can have different fields.
- Flexible: It can integrate with many modern apps. [Python, Node]
- Scalable: Can handle massive amount of data (unstructured).
- cloud ready: It has ATLAS for cloud hosting & management.
- High-Performance: It is faster than other query languages.



## ■ Key components:

- → Mongod
  - ↳ It is the main MongoDB Server process.
  - ↳ It controls databases, storages, queries in the backend.
  - ↳ It listens on a port (default 27017) for client connections.
- → Mongosh
  - ↳ It is the MongoDB shell
  - ↳ You run commands and all queries here
- → MongoDB Compass →
  - ↳ It is visualization tool.
  - ↳ It is a GUI tool for visualizing data.

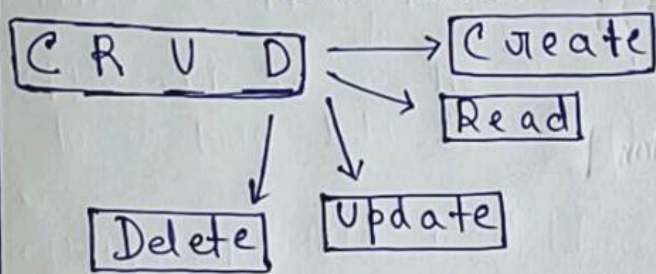
## ■ How to install MongoDB?

You can simply install it through the official MongoDB website.

————— X —————



## 1. Basic operations:



1. CREATE → For inserting data by using **INSERT** Command.
2. READ → For retrieve the data by using **SELECT** Command.
3. UPDATE → For updating the existing value by using **UPDATE** Command.
4. DELETE → For deleting the existing value in a database by using **DELETE** Command.

## Basic Commands

\* In mongoDB shell → you can give commands.

\* Mongo Compass → Mongo DB shell → commands

### A) commands:

→ USE myDatabase ← It will switch to your specific database.

→ Show dbs ← It will show all available databases.

→ Show collections ← It will show all collections available inside that specific database only.

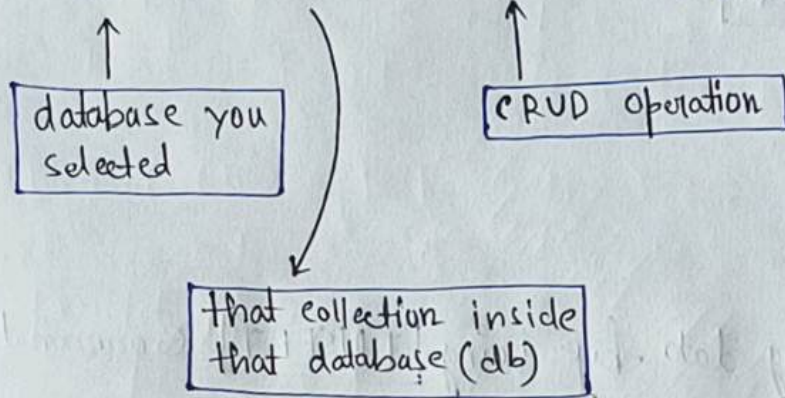
→ db.collection.insertOne({ })



## CREATE / INSERT

④ INSERTING single value into the collection.

```
db.collections.insertOne({});
```

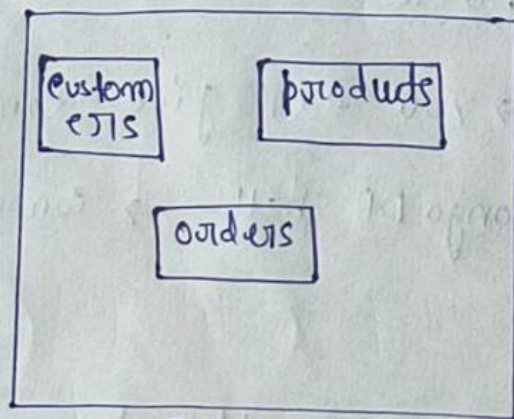


⑤ Inserting multiple values into the collection.

```
db.collections.insertMany([{}]);
```

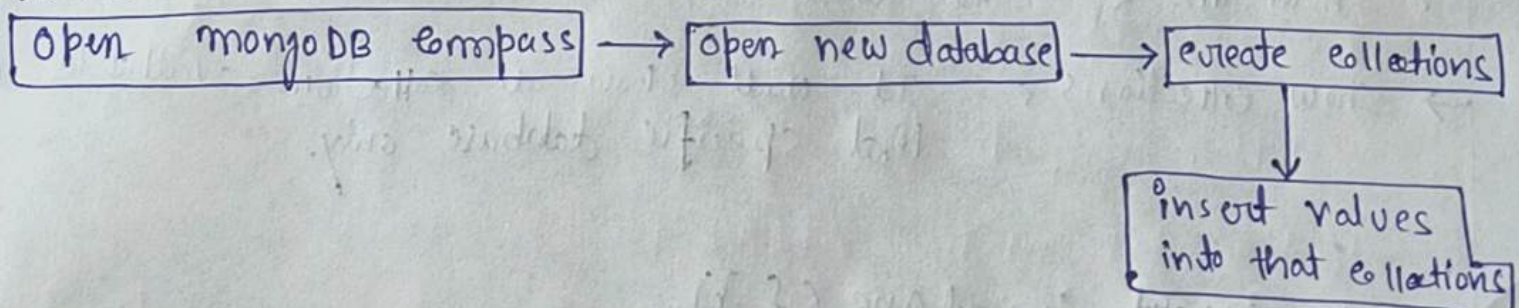
### Example:

① Suppose we made a database E-commerce and in that database we are creating many collections.



ecommerce

### steps:





② In mongDB shell:

> show dbs

> show collections

> use E-commerce

> db

> db.products.insertMany ([ {

"product\_name": "A",

"price": 1234,

"product\_rate": 3

},

{

"product\_name": "B",

"price": 5678,

"product\_rate": 2

},

{

"product\_name": "C",

"price": 8798,

"product\_rate": 5

}] );

```
> db.customers.insertMany([
```

```
  "name": "Rahul",
```

```
  "address": "Kolkata",
```

```
  "ph-no": "6289449298"
```

```
},
```

```
{
```

```
  "name": "Rakesh",
```

```
  "address": "Durgam",
```

```
  "ph-no": "6274125233"
```

```
},
```

```
{
```

```
  "name": "Aqinab",
```

```
  "address": "Assansole",
```

```
  "ph-no": "9574580679"
```

```
]);
```

■ You can also drop database permanently.

```
db.dropDatabase()
```

By that command, it will be delete properly with it's all collections.

■ Syntax:

```
db.collections.insertOne({ "Key" : "Value" })
```

```
db.collections.insertMany([ { "Key" : "Value" } ])
```



## ① Syntax: Find Query

db.collections.find() ← It will return all Key:Value pairs

db.collections.find({"Key": "Value"}) ← {It will return a specific information about the collection using "Key": "Value"}

db.collections.find().pretty()

## ■ Example

db.products.find({"product-id": 12})

db.orders.find({"order-id": 1})

db.customers.find({"customer-id": 123})

## ② Syntax: Comparison operators:

◆ db.products.find({"price": {"\$gt": 500}})

◆ db.products.find({"price": {"\$gte": 500, "\$lte": 1000}})

→ \$gt → greater than.

→ \$lt → less than.

→ \$gte → greater than or equal to.

→ \$lte → less than or equal to.

## ③ Syntax: Logical operators:

◆ db.products.find({"\$or": [{"category": "computer"}, {"stock": {"\$gte": 500, "\$lte": 600}}]})

◆ db.products.find({"\$and": [{"category": "Phone"}, {"stock": 180}]}))



- $\$ or \rightarrow$  one condition must be true between two condition.
- $\$ and \rightarrow$  All conditions must be same.

### Projection (Select Specific Field)

#### ■ Syntax:

`db.orders.find( {}, { "user": 1, "products": 1, "_id": 0 } )`

### Sorting & Limiting

#### ■ Syntax:

- ◆ `db.products.find().sort( { "price": 1 } ).limit(1)`
- ◆ `db.products.find().sort( { "price": -1 } ).limit(1)`
- ◆ `db.products.find().sort( { "price": 1 } ).skip(1).limit(1)`

- 1  $\rightarrow$  Ascending order
- -1  $\rightarrow$  Descending order
- skip  $\rightarrow$  How many rows or values have to skip
- limit  $\rightarrow$  How many rows we want.



## UPDATE

### ① Update one :

By using updateone command you can update only one value of any document.

#### ■ Basic Command:

```
db.collection.updateOne(  
  { "name": "A" },  
  { $set : {  
    "salary": 12000  
  }}  
)
```

### ② Update Many:

By using update Many we can update multiple values in a document.

#### ■ Basic Command:

```
db.collection.updateMany(  
  { "course": "BEA" },  
  { $set : {  
    "year": 3  
  }}  
)
```



### ③ Replace:

By using Replace, we can replace many docs.

### ■ Basic Command:

```
db.collection.replaceOne(
```

```
  { "name": "A" },
```

```
  { "age": 18, "Course": "MI" }
```

```
)
```

### ④ increase:

By using \$inc, we can increase values of any document.

```
db.collection.updateOne(
```

```
  { "name": "B" },
```

```
  { $inc: {
```

```
    "Salary": 18000 }
```

```
  }
```

```
)
```



# DELETE

①

## Delete One:

By using DeleteOne we can easily delete one specific value of any document.

■

## Basic Command:

```
db.collection.deleteOne(
```

```
{ "name": "A" }
```

```
)
```

②

## Delete Many:

By using delete many, we can easily delete many value in a document.

■

## Basic Command:

```
db.collection.deleteMany(
```

```
{ "category": "Electronics" }
```

```
)
```

## ■ Push :

By using push command, we can easily add new value in a array of any document.

## ■ Basic Command:

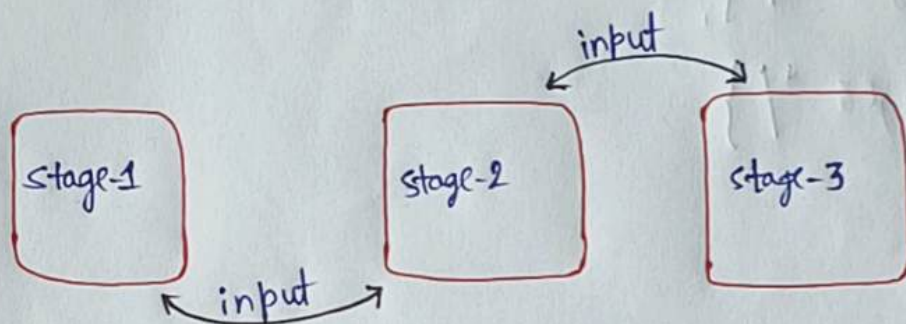
```
db. collections. update Many (
  { "name": "Mouse" },
  { $push: {
    "tags": "new"
  }
})
```



# Aggregation Pipeline

Defn

Aggregation Pipeline is use to transform and analyze data. They work in stages. One stage is dependent on the previous stage. The output of one stage becomes the input of next stage.



→ Suppose,

→ you want to extract student details.

→ stage-1:

- Find school name (specific).

→ stage-2:

- Find the class of that specific student.

→ stage-3:

- Find the specific id of that student.

∴ By following this steps/stages you can easily find one specific detail.

## Syntax

```
db.collection.aggregate([
```

```
  { /* stage1 */ },
```

```
  { /* stage2 */ },
```

```
  { /* stage3 */ },
```

```
]);
```

① Example → ① : [Filter docs (\$match)] :

```
db.Sales.aggregate([
```

```
  { $match : { category: "Foruit" } }
```

```
]);
```

② Example → ② : [Filter docs (\$project)] :

```
db.Sales.aggregate([
```

```
  { $match : { category: "Foruit" } }
```

```
  { $project : { _id: 0, price: 1, quantity: 1 } }
```

```
]);
```



⑤ Example → ⑤ [ \$match + \$group ]

db.sales.aggregate([

{ \$match: { "category": "Fruits" },

{ \$group: {

\_id: null,

totalFruits: { \$sum: {

\$multiply: [ "\$price", "\$quantity" ]

}}

}

}

]);

③ Example → ③ [Filter docs (grouping [\$group])]

db.sales.aggregate([

{ \$group : {

\_id: "\$category",

totalSales: { \$sum: { \$multiply: [ "price", "quan" ]

}}}

}}

]);

④ Example → ④ [Sorting (\$sort)]

db.sales.aggregate([

{ \$group : {

\_id: "\$category",

total : {

\$sum : "\$ price"

}

}},

{ \$sort : {

total : -1

}}

]);



## Indexes

Defn

- Indexes in MongoDB are the special kind of data structures which is used to improve read & write operations.
- It actually works like the index page of a book.  
↳ you can find specific field of a collection easily

Create Index:

```
db.collection.createIndex( { name: 1 } )
```

// Ascending index on "name" field.

View Index:

```
db.products.getIndex()
```

// List all indexes on 'products' collection.

————— × —————